

```
===== Program Code =====
```

```
import matplotlib
matplotlib.use('Agg')
```

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

```
# =====
```

```
# Helper function for saving with title
```

```
# =====
```

```
def save_with_title(image, filename, title, cmap=None):
    plt.figure()
    if cmap:
        plt.imshow(image, cmap=cmap, vmin=0, vmax=255)
    else:
        plt.imshow(image)
    plt.title(title)
    plt.axis("off")
    plt.savefig(filename, bbox_inches="tight", pad_inches=0.2)
    plt.close()
```

```
# =====
```

```
# Step 1: Load Image
```

```
# =====
```

```
img = Image.open("daylilies.jpg").convert("RGB")
ImJPG = np.array(img)
```

```
m, n, l = ImJPG.shape
```

```
print("Shape:", ImJPG.shape)
print("m =", m)
print("n =", n)
print("l =", l)
```

```
save_with_title(ImJPG, "step1_original.png", "Step 1: Original Image")
```

```
# =====
```

```
# Step 2: RGB Channels (grayscale)
```

```
# =====
```

```
redChannel = ImJPG[:, :, 0]
greenChannel = ImJPG[:, :, 1]
blueChannel = ImJPG[:, :, 2]
```

```
save_with_title(redChannel, "step2_redChannel.png", "Step 2: Red Channel", cmap="gray")
save_with_title(greenChannel, "step2_greenChannel.png", "Step 2: Green Channel", cmap="gray")
save_with_title(blueChannel, "step2_blueChannel.png", "Step 2: Blue Channel", cmap="gray")
```

```
# =====
```

```
# Step 3: Grayscale (matrix method)
```

```
# =====
```

```
GrayMatrix = np.array([
    [1/3, 1/3, 1/3],
    [1/3, 1/3, 1/3],
    [1/3, 1/3, 1/3]
])
```

```
pixels = ImJPG.astype(float).reshape(-1, 3)
```

```
gray_pixels = pixels @ GrayMatrix.T
gray_pixels = np.clip(gray_pixels, 0, 255).astype(np.uint8)
```

```
ImJPG_Gray = gray_pixels.reshape(m, n, 3)

save_with_title(ImJPG_Gray, "step3_grayscale.png", "Step 3: Grayscale Image")

# =====
# Step 4: Sepia (matrix method)
# =====

SepiaMatrix = np.array([
    [0.393, 0.769, 0.189],
    [0.349, 0.686, 0.168],
    [0.272, 0.534, 0.131]
])

sepia_pixels = pixels @ SepiaMatrix.T
sepia_pixels = np.clip(sepia_pixels, 0, 255).astype(np.uint8)

ImJPG_Sepia = sepia_pixels.reshape(m, n, 3)

save_with_title(ImJPG_Sepia, "step4_sepia.png", "Step 4: Sepia Image")

# =====
# Step 5: Red-only filter
# =====

RedMatrix = np.array([
    [1, 0, 0],
    [0, 0, 0],
    [0, 0, 0]
])

red_pixels = pixels @ RedMatrix.T
red_pixels = np.clip(red_pixels, 0, 255).astype(np.uint8)

ImJPG_Red = red_pixels.reshape(m, n, 3)

save_with_title(ImJPG_Red, "step5_redFilter.png", "Step 5: Red Filter Image")

# =====
# Step 6: Color permutation
# =====

PermuteMatrix = np.array([
    [0, 0, 1],
    [0, 1, 0],
    [1, 0, 0]
])

perm_pixels = pixels @ PermuteMatrix.T
perm_pixels = np.clip(perm_pixels, 0, 255).astype(np.uint8)

ImJPG_Perm = perm_pixels.reshape(m, n, 3)

save_with_title(ImJPG_Perm, "step6_permuted.png", "Step 6: Color Permutation")

# =====
# Step 7: Determinants (invertibility check)
# =====

print("\nStep 7: Determinants")

print("det(GrayMatrix) =", np.linalg.det(GrayMatrix))
print("det(SepiaMatrix) =", np.linalg.det(SepiaMatrix))
print("det(RedMatrix) =", np.linalg.det(RedMatrix))
print("det(PermuteMatrix) =", np.linalg.det(PermuteMatrix))
```

```
# =====
# Step 8: Recover original using inverse
# =====

PermuteInverse = np.linalg.inv(PermuteMatrix)

recovered_pixels = perm_pixels.astype(float) @ PermuteInverse.T
recovered_pixels = np.clip(recovered_pixels, 0, 255).astype(np.uint8)

ImJPG_Recovered = recovered_pixels.reshape(m, n, 3)

save_with_title(ImJPG_Recovered, "step8_recovered.png", "Step 8: Recovered Image")

===== Program Output =====
Shape: (308, 322, 3)
m = 308
n = 322
l = 3

Step 7: Determinants
det(GrayMatrix) = 0.0
det(SepiaMatrix) = 1.2099999999999696e-07
det(RedMatrix) = 0.0
det(PermuteMatrix) = -1.0
```